

Django

TomerBu

נושאים להיום:

המשך עבודה עם טפסים
טיפוסים במסד הנתונים
יחסי גומלין

המשך עבודה על פרוייקט משיעור קודם - lec2

טרמינל:

```
python manage.py runserver
```

דפדפן:

<http://127.0.0.1:8000/quotes/>

תבנית לשימוש חוזר בhtml:

lec2/quotes/templates/quotes/list.html

העתקה של base.html

```
{% extends 'quotes/base.html' %}
```

```
{% block title %}  
Quotes  
{% endblock %}
```

← דריסה של הבלוק title

```
{% block content %} ← דריסה של הבלוק content
```

```
<h1>Quotes List</h1>
```

```
<table class="table table-striped table-bordered">
```

```
<tr>
```

```
<th>Quote</th>
```

```
<th>Author</th>
```

```
<th>Year</th>
```

```
</tr>
```

```
{% for quote in quotes %}
```

```
<tr>
```

```
<td>{{ quote.quote }}</td>
```

```
<td>{{ quote.author }}</td>
```

```
<td>{{ quote.year }}</td>
```

```
</tr>
```

```
{% endfor %}
```

```
</table>
```

```
{% endblock %}
```

טופס להוספת Quote - טופס html

lec2/quotes/**templates/quotes/add.html**

```
{% extends 'quotes/base.html' %}

{% block title %}
    Add a Quote
{% endblock %}

{% block content %}
    <h1>Add A Quote</h1>

    <form method="post">
        {% csrf_token %}
        <div class="form-group">
            <label for="quote">Quote</label>
            <textarea rows="2" class="form-control" name="quote" id="quote" placeholder="Enter Quote"></
textarea>
        </div>

        <div class="form-group">
            <label for="author" class="form-label">Author</label>
            <input type="text" class="form-control" name="author" id="author" placeholder="Enter Author" / >
        </div>

        <div class="form-group">
            <label for="year">Year</label>
            <input type="number" class="form-control" name="year" id="year" placeholder="Enter Year" />
        </div>

        <button type="submit" class="btn btn-primary">Submit</button>
    </form>
{% endblock %}
```

פונקצית View : להוספת Quote

lec2/quotes/**views.py**

```
def add_quote(req: HttpRequest):  
    if req.method == 'POST':  
        author = req.POST.get('author')  
        quote = req.POST.get('quote')  
        year = req.POST.get('year')  
  
        if author and quote and year:  
            quote = Quote(author=author, quote=quote, year=year)  
            quote.save()  
            return redirect('quotes')  
        else:  
            return render(req, 'quotes/add.html', {'error': 'must fill all the fields'})  
  
    return render(req, 'quotes/add.html')
```

ניתוב - url:

lec2/quotes/**urls.py**

```
from django.urls import path
from quotes.views import add_quote, get_quotes

urlpatterns = [
    path('', get_quotes, name='quotes'),
    path('add', add_quote, name='add_quote')
]
```

<http://localhost:8000/quotes/add>

Site Header

Add A Quote

Quote

Author

Year

מחיקה של רשומה לפי id:

quotes/urls.py

```
from django.urls import path
from quotes.views import add_quote, get_quotes, delete_quote

urlpatterns = [
    # /quotes/
    path('', get_quotes, name='quotes'),

    # /quotes/add
    path('add', add_quote, name='add_quote'),

    # /quotes/delete/2
    path('delete/<int:id>', delete_quote, name='delete_quote'),
]
```

quotes/views.py

```
def delete_quote(req: HttpRequest, id):
    quote = Quote.objects.get(id=id)
    quote.delete()
    return redirect('/quotes/')
```


מחיקה של רשומה לפי id:

quotes/templates/quotes/list.html

```
{% extends 'quotes/base.html' %}

{% block title %}
Quotes
{% endblock %}

{% block content %}
<h1>Quotes List</h1>
<table class="table table-striped table-bordered">
  <tr>
    <th>Quote</th>
    <th>Author</th>
    <th>Year</th>
    <th>Actions</th>
  </tr>

  {% for quote in quotes %}
    <tr>
      <td>{{ quote.quote }}</td>
      <td>{{ quote.author }}</td>
      <td>{{ quote.year }}</td>
      <td>
        <a href="{% url 'delete_quote' quote.id %}">Delete</a>
      </td>
    </tr>
  {% endfor %}
</table>
{% endblock %}
```

שינוי של כתובת בקובץ urls.py לא ישפיע על הhtml

עדכון רשומה לפי id:

(1) ניצור קובץ html לטופס עריכה

(2) ניצור פונקציית view: בדומה לפעולה של add

(3) נוסף url בקובץ urls.py

(4) נוסף קישור בקובץ list.html

עדכון רשומה לפי id:

```
{% extends 'quotes/base.html' %}

{% block title %}
    Edit Quote
{% endblock %}

{% block content %}
    <form method="post">
        {% csrf_token %}
        <div class="form-group">
            <label for="quote">Quote</label>
            <textarea rows="2" class="form-control" name="quote" id="quote" placeholder="Enter Quote">

                {{quote.quote}}

            </textarea>
        </div>

        <div class="form-group">
            <label for="author" class="form-label">Author</label>
            <input value="{{quote.author}}" type="text" class="form-control" name="author" id="author" placeholder="Enter
Author" />
        </div>

        <div class="form-group">
            <label for="year">Year</label>
            <input value="{{quote.year}}" type="number" class="form-control" name="year" id="year" placeholder="Enter Year" />
        </div>

        <button type="submit" class="btn btn-primary">Submit</button>
    </form>
{% endblock %}
```

עדכון רשומה לפי id:

quotes/views.py

```
def edit_quote(req: HttpRequest, id):  
  
    quote = Quote.objects.get(id=id)  
  
    if req.method == "GET":  
        return render(req, "quotes/edit.html", {"quote": quote})  
  
    quote.author = req.POST.get('author', quote.author)  
    quote.quote = req.POST.get('quote', quote.quote)  
    quote.year = req.POST.get('year', quote.year)  
    quote.save()  
    return redirect('/quotes/')
```

עדכון רשומה לפי id:

quotes/urls.py

```
from django.urls import path
from quotes.views import add_quote, get_quotes, delete_quote, edit_quote

urlpatterns = [
    # /quotes/
    path('', get_quotes, name='quotes'),

    # /quotes/add
    path('add', add_quote, name='add_quote'),

    # /quotes/delete/2
    path('delete/<int:id>', delete_quote, name='delete_quote'),

    # /quotes/edit/3
    path('edit/<int:id>', edit_quote, name='edit_quote'),
]
```

```
<a href="{% url 'edit_quote' quote.id %}">Edit</a>
```

התמודדות עם 404

quotes/views.py

```
def delete_quote(req: HttpRequest, id):  
  
    try:  
        quote = Quote.objects.get(id=id)  
        quote.delete()  
        return redirect('/quotes/')  
    except Quote.DoesNotExist:  
        return render(req, 'quotes/404.html', status=404)  
  
def edit_quote(req: HttpRequest, id):  
    try:  
        quote = Quote.objects.get(id=id)  
  
        if req.method == "GET":  
            return render(req, "quotes/edit.html", {"quote": quote})  
  
        quote.author = req.POST.get('author', quote.author)  
        quote.quote = req.POST.get('quote', quote.quote)  
        quote.year = req.POST.get('year', quote.year)  
        quote.save()  
        return redirect('/quotes/')  
    except Quote.DoesNotExist:  
        return render(req, 'quotes/404.html', status=404)
```

מסד נתונים:

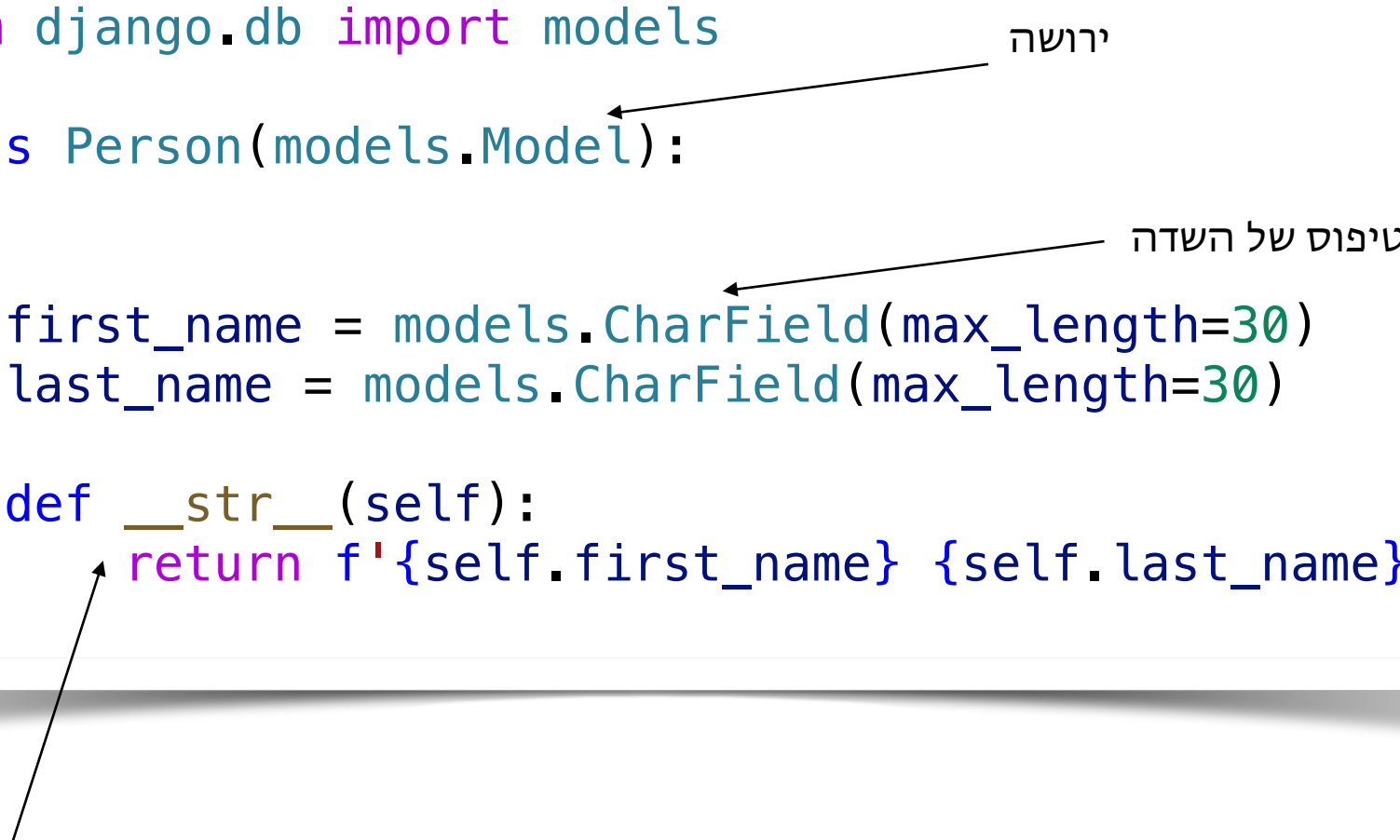
מודל עבור הנתונים:

```
from django.db import models

class Person(models.Model):

    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)

    def __str__(self):
        return f'{self.first_name} {self.last_name}'
```



הצגה ידידותית בממשק הניהול

מסד נתונים:

```
class Quote(models.Model):
    # props:
    author = models.CharField(max_length=60)
    quote = models.TextField()
    year = models.IntegerField()

    id שדה עבור ה

    # admin timestamps
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f'{self.quote} - {self.author} ({self.year})'
```


מסד נתונים:

פעולות בסיסיות:

```
from quotes.models import Quote
```

פעולות קריאה של כל האובייקטים:

get all:

get all order by:

```
#SELECT * FROM Quotes  
quotes = Quote.objects.all()
```

```
#SELECT * FROM Quotes ORDER BY year  
quotes = Quote.objects.all().order_by('year')  
quotes = Quote.objects.all().order_by('-year')
```

פעולת קריאה לפי id

פעולת קריאה לפי פרמטר

```
quote = Quote.objects.get(id=id)
```

מחיקה:

```
quote = Quote.objects.get(id=id)  
quote.delete()
```

יצירה:

```
quote = Quote(quote="", author="", year=1999)  
quote.save()
```

עריכה:

```
quote = Quote.objects.get(id=id)  
quote.year = 1989  
quote.save()
```

מסד נתונים:

בדטה חשוב מקום בדיסק: אם אפשר לחסוך - חוסכים

טיפוסים מספריים:

סוג שדה	הסבר	פרמטרים חשובים
AutoField	(Integer) מזהה ייחודי שנוצר אוטומטית	primary_key=True
BigAutoField	(BigInteger) מזהה ייחודי גדול יותר	primary_key=True
BigIntegerField	שדה מספר שלם גדול במיוחד	
IntegerField	שדה מספר שלם	
PositiveIntegerField	שדה למספרים שלמים חיוביים בלבד	
PositiveBigIntegerField	שדה למספרים חיוביים גדולים בלבד	
SmallIntegerField	שדה למספר שלם קטן	
PositiveSmallIntegerField	שדה למספרים שלמים קטנים חיוביים בלבד	
DecimalField	שדה למספרים עשרוניים מדויקים	max_digits, decimal_places
FloatField	(float) שדה למספרים ממשיים	

```
class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
    age = models.Sm
def __str__(self):
    return f'{s
PositiveSmallIntegerField
```

דוגמא:

מספרים
מחרוזות
תאריכים
ועוד

מסד נתונים:

בדטה חשוב מקום בדיסק: אם אפשר לחסוך - חוסכים

סוג שדה	הסבר	פרמטרים חשובים
CharField	שדה טקסט עם אורך מוגבל	max_length
TextField	שדה טקסט ללא מגבלת אורך	
SlugField	טקסט ידידותי ל-URL (אותיות, מספרים, מקפים)	max_length, allow_unicode
EmailField	טקסט המאמת כתובת דוא"ל	
URLField	טקסט המאמת כתובת URL	
UUIDField	מזהה ייחודי בפורמט UUID	
JSONField	שדה לאחסון נתוני JSON	

מספרים
מחרוזות
תאריכים
ועוד

מסד נתונים:

טיפוסים לתאריכים:

סוג שדה	הסבר	פרמטרים חשובים
DateField	שדה לתאריך בלבד	auto_now, auto_now_add
DateTimeField	שדה לתאריך ושעה	auto_now, auto_now_add
TimeField	שדה לשעה בלבד	
DurationField	שדה למרווחי זמן (timedelta)	

בעת כתיבת רשומה חדשה (add) ישמור את התאריך הנוכחי.
`created_at = models.DateTimeField(auto_now_add=True)`

בכל עריכת רשומה (update) ישמור את התאריך הנוכחי.
`updated_at = models.DateTimeField(auto_now=True)`

מסד נתונים:

טיפוסים ליחסי גומלין:

סוג קשר	פרמטרים חשובים	ייצוג במסד נתונים	הסבר
ForeignKey	on_delete, related_name, related_query_name, to_field	עמודת מפתח זר בטבלה, המקשרת לטבלת היעד.	קשר *רבים-לאחד* בין שני מודלים.
ManyToManyField	related_name, related_query_name, through, symmetrical	טבלת ביניים (junction table) המקשרת בין שתי הטבלאות.	קשר *רבים-לרבים* בין שני מודלים.
OneToOneField	on_delete, related_name, related_query_name, to_field	עמודת מפתח ייחודי עם מפתח זר בטבלה.	קשר *אחד-לאחד* בין שני מודלים.

מסד נתונים:

One to One

קשר של אחד לאחד:

User

email
password

UserDetails

address
phone1

לכל משתמש יש שורה בטבלת פרטי משתמש
פרטי משתמש שייכים למשתמש אחד בלבד

מסד נתונים:

One to Many

קשר של אחד לרבים:

הזמנה

הזמנה א

פרטי הזמנה

3 במבה

2 ביסלי

1 אבטיח

בהזמנה אחת יש הרבה פריטים

מסד נתונים:

Many To Many

קשר של רבים לרבים:

סרט

ז'אנר

ספיידרמן

באטמן

scifi
action
thriller
drama

בסרט יש הרבה ז'אנרים
בזאנר יש הרבה סרטים

מסד נתונים:

ספק

חנות

מוצר

קטגוריות
(מוצרי חלב, ירקות, פירות)

הזמנה

פרטי הזמנה

לקוח

כתובת

מסד נתונים:



id	שם החנות	כתובת
1	סופרסופר	רחוב הנביאים 4, בני ברק, ישראל 85950

אפשר לשמור כתובת כשורה בדטה-בייס
זה תלוי בסוג העסק שצריך את הדטה-בייס

אם נפרק את הכתובת לפרטי כתובת:
נוכל לבצע שאילתות ביעילות לגבי:
עיר, רחוב, וארץ

מסד נתונים:

כתובת:

חנות:

id	שם החנות	כתובת
1	סופרסופר	4

טבלת כתובות - דוגמה לטבלת SQL

ID	רחוב	מספר	עיר	מיקוד	ארץ
1	אחד העם	15	תל אביב	6473915	ישראל
2	דיזנגוף	125	תל אביב	6473820	ישראל
3	הרצל	42	ירושלים	9414906	ישראל
4	בן גוריון	88	חיפה	3109601	ישראל
5	ויצמן	7	רחובות	7610001	ישראל

ניצור אפליקציה חדשה לטובת הדוגמאות:

```
python manage.py startapp api
```

lec2/api/views.py

```
from django.shortcuts import render
from django.http import HttpResponse

# Create your views here.

def index(request):
    return HttpResponse("Hello, API!")
```

lec2/lec2/settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'quotes',
    'bootstrap5',
    'api'
]
```

lec2/lec2/urls.py

```
from django.urls import path

from api.views import index

urlpatterns = [
    path('', index, name='index'),
]
```

בתוך תיקיית api שנוצרה - ניצור קובץ urls.py

ניצור אפליקציה חדשה לטובת הדוגמאות:

נעדכן את הurls של הפרויקט:

lec2/lec2/urls.py

```
from django.contrib import admin
from django.urls import path, include
from quotes.urls import urlpatterns as quote_patterns

urlpatterns = [
    path('admin/', admin.site.urls),
    path('quotes/', include(quote_patterns)),
    path('api/', include('api.urls'))
]
```

python manage.py runserver

מודל ראשון:

lec2/api/models.py

```
from django.db import models

class Address(models.Model):

    street = models.CharField(max_length=100)
    apartment = models.CharField(max_length=10, blank=True)
    city = models.CharField(max_length=50)
    zip_code = models.CharField(max_length=10)
    country = models.CharField(max_length=50, default='IL')

    def __str__(self):
        return f"{self.street}, {self.city}, {self.apartment} {self.zip_code}, {self.country}"
```

נוסיף את המודל לממשק הניהול של API:

lec2/api/admin.py

```
from django.contrib import admin

from api.models import Address

# Register your models here.
admin.site.register([Address])
```

<http://localhost:8000/admin/>

מודל ראשון:

```
python manage.py makemigrations api
```

```
python manage.py migrate
```

```
python manage.py runserver
```

<http://localhost:8000/admin/>

נרצה ליצור מודל לחנות: models.py

(1) ניצור מחלקה בmodels.py

id	שם החנות	כתובת
1	סופרסופר	4

(2) נוסיף את המודל שלנו לרשימה בקובץ admin.py במודול api

(3) נריץ מיגרציות

(4) נסתכל בממשק ניהול שהכל עובד כרצוי

<http://localhost:8000/admin/>

מודל לחנות:

models.py

```
class Store(models.Model):
    name = models.CharField(max_length = 100)
    address = models.OneToOneField(Address, on_delete=models.CASCADE)

    def __str__(self):
        return f"{self.name} - {self.address.city}, {self.address.street}"
```

admin.py

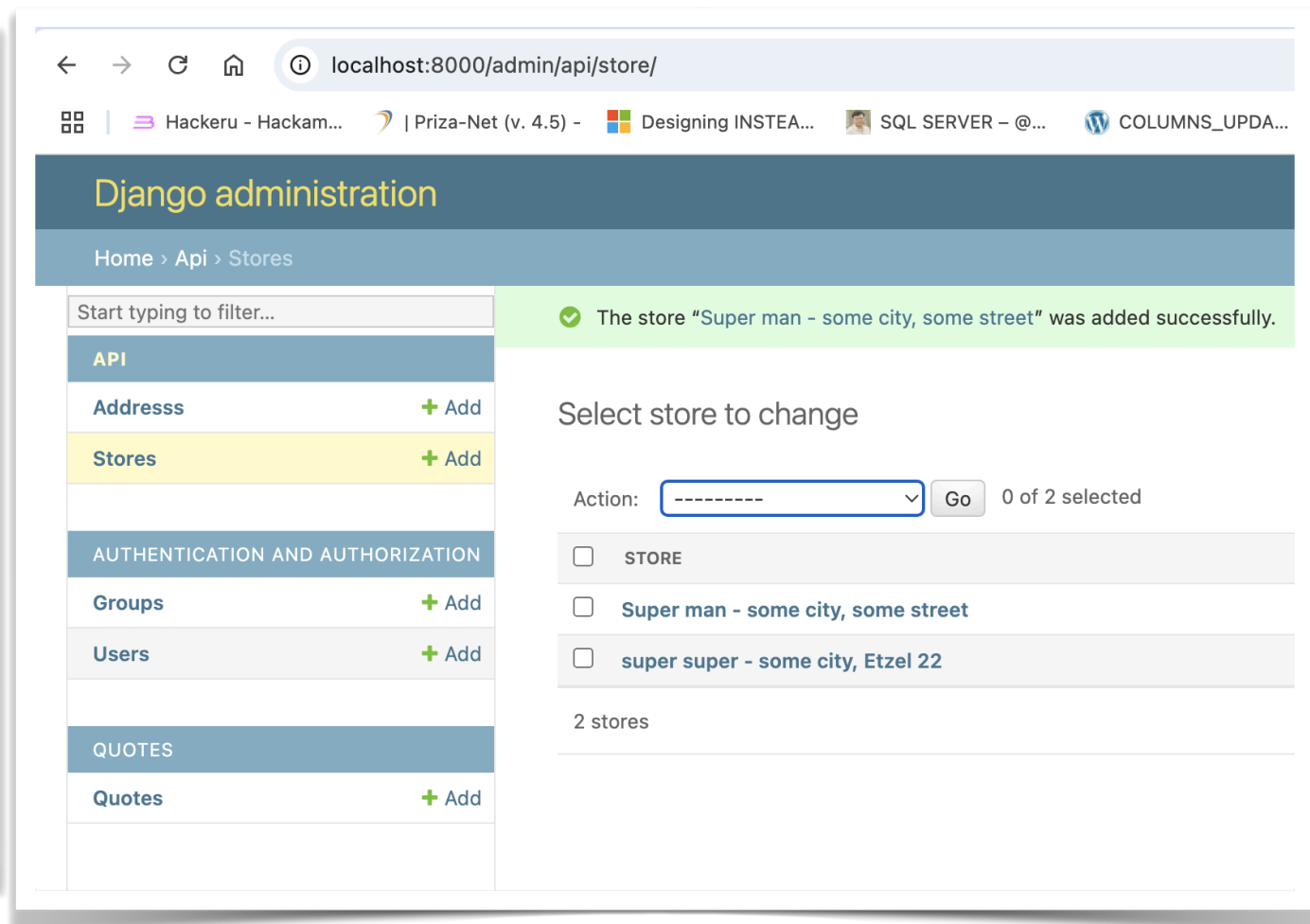
```
from django.contrib import admin

from api.models import Address, Store

# Register your models here.
admin.site.register([Address, Store])
```

#####

```
python manage.py makemigrations api
python manage.py migrate
python manage.py runserver
#####
```



אחרי ההפסקה:

נסתכל על מסד הנתונים

ניצור קשרי גומלין של אחד אחד

ניצור קשרי גומלין של אחד לרבים

ניצור קשרי גומלין של רבים לרבים

נסתכל על מסד הנתונים

קטגוריות (מוצרי חלב, ירקות, פירות)

מוצר

שם המוצר
תיאור
מחיר

יחסי גומלין אחד לרבים (למוצר יש קטגוריה אחת) בקטגוריה יש הרבה מוצרים
יחסי גומלין לחנות

אחד לרבים:

```
class Category(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name

class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    description = models.TextField()

    # Foreign key = OneToMany
    category = models.ForeignKey(Category, on_delete=models.CASCADE)
    store = models.ForeignKey(Store, on_delete=models.CASCADE)

    def __str__(self):
        return self.name
```

מוצר שייך לחנות אחת
לחנות יש הרבה מוצרים

אחד לרבים

מוצר שייך לקטגוריה אחת
בקטגוריה יש הרבה מוצרים

אחד לרבים

רבים לרבים:

ספק

חנות

לחנות יכולים להיות הרבה ספקים

ספק עובד עם הרבה חנויות

```
class Supplier(models.Model):  
    name = models.CharField(max_length=100)  
    contact_email = models.EmailField()  
    stores = models.ManyToManyField(Store)  
  
    def __str__(self):  
        return self.name
```

המקרה הפשוט של רבים לרבים

רבים לרבים:

```
from django.contrib import admin

from api.models import Address, Store, Category, Product, Supplier

# Register your models here.
admin.site.register([Address, Store, Category, Product, Supplier])


"""
python manage.py makemigrations api && python manage.py migrate && python manage.py
runserver
"""
```

אחד לאחד:

כתובת

Rows: 2

	id	street	apartment	city	zip_code	country
1	1	some street	2	some city	123123	IL
2	2	Etzel 22	2	some city	123125	IL

id	name	address_id
1	super super	2
2	Super man	1

עמודת מפתח
Primary key

מפתח זר
Foreign key

חנות

בשפת SQL

אפשר לאסוף מידע מכמה טבלאות יחד
ולהציג דוח בטבלה מסכמת

אחד לאחד:

אחת הטבלאות מוסיפה עמודה של מפתח זר לטבלה האחרת.

אחד לרבים:

אחת הטבלאות מוסיפה עמודה של מפתח זר לטבלה האחרת.

Primary Key

id	name
1 Dairy Products	
2 Fruits	
3 Vegetables	

Primary Key

id	name	price	descripti...	categ...
1	Apple	10	Per Kg	2
2	Milk	7	Per unit	1

Foreign Key

רבים לרבים:

primary key

store

id	name
1	super super
2	Super man

חנות:

primary key

supplier

id	name	contact_email
1	FruitsDotCom	FruitsDotCom@gmail.com
2	Veggies	Veggies@gmail.com

ספק:

primary key

primary key

id	supplier_id	store_id
1	1	1
2	1	2
3	2	2

חנויות_ספקים:

שיעורי בית:

צרו את המודלים הבאים:

Category:
name

שם הקטגוריה - טקסט קצר

קטגוריות לדוגמא:

תכנות, פייתון, לולאות, HTTP, וכו'...

Post:
title
content
created
modified
categories

קשר של רבים לרבים
ManyToManyField

Author:
name
birthday
user

קשר של אחד לאחד

Comment:
author
text
created
post

קשר של אחד לרבים:
models.ForeignKey

קשר של אחד לרבים:
models.ForeignKey

מלאו ערכים בadmin

תרגיל רשות:

צרו עמוד HTML שמציג את כל הכתבות
צרו עמוד HTML ליצירת כתבות
צרו עמוד HTML שעורך כתבה

לכל עמוד יש ליצור:

url

view

קובץ html

התקנות לשיעור הבא:

Postgres

<https://www.postgresql.org/download>